Old Process= use BPMN (analysis of the existing system)

New Process= use **UML Activity Diagram**

UML Activity Diagram: best for explaining how a software system algorithm, or application feature behaves.

- Model software behavior and activity flow
- Main focus- Actions, logic, data flow, and system behavior

BPMN: best for describing **an end-to-end business process involving people, offices, organizations, events, documents, and information systems.**

The simplest distinction is:

BPMN models how the organization delivers the process; UML Activity Diagrams model how the software or use case behaves <u>within that process.</u>

For example:

1. Use **BPMN** to document the complete government application and approval process.
    - Identify the activities performed by the information system.
2. Use a **UML activity diagram** to describe the internal behavior of each automated activity.

Diagramming a Software Process Using a UML Activity Diagram

A **UML Activity Diagram** explains how a software system, algorithm, or application feature behaves. It focuses on:

- Actions performed by the user or system;
- Logical sequence of operations;

- Decisions and conditions;
- Repetition or loops;
- Data entering and leaving an action;
- Parallel activities; and
- Successful and exceptional outcomes.

A calculator is a useful example because it contains input, validation, decisions, computation, error handling, output, and repetition.

1. Define the process before drawing

Before choosing symbols, establish the process boundary.

Example: Basic calculator

Process name: Perform a Basic Calculation

Objective: Allow a user to enter two numbers, select an arithmetic operation, and obtain a result.

Start condition: The calculator is opened.

Inputs:

- First operand;
- Operator: addition, subtraction, multiplication, or division;
- Second operand.

Output:

- Calculated result; or
- Error message.

Business and system rules:

1. Both operands must be valid numbers.
2. The selected operator must be supported.
3. Division by zero is prohibited.
4. The user may perform another calculation after receiving a result.
5. Invalid input should return the user to data entry.

End condition: The user chooses not to perform another calculation.

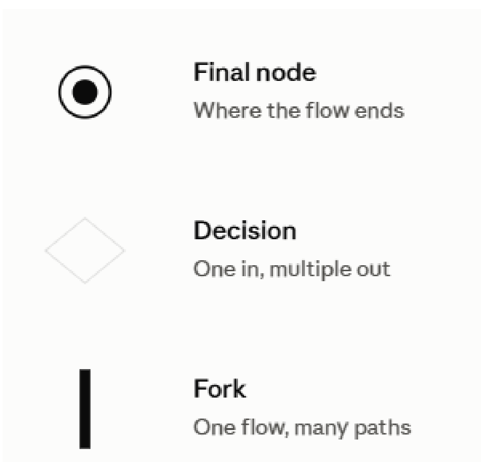
Defining these items first prevents the diagram from becoming incomplete or inconsistent.

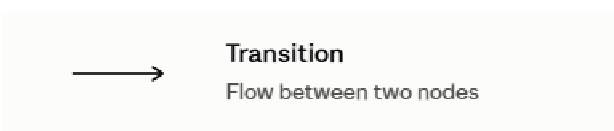
2. Identify the process activities

Write the process initially as a simple sequence:

1. Display the calculator interface.
2. Enter the first operand.
3. Select an operator.
4. Enter the second operand.
5. Validate the inputs.
6. Check for division by zero.
7. Perform the selected operation.
8. Display the result.
9. Ask whether another calculation will be performed.
10. Repeat or terminate the process.

These steps are then converted into UML actions, decisions, flows, object nodes, and final nodes.

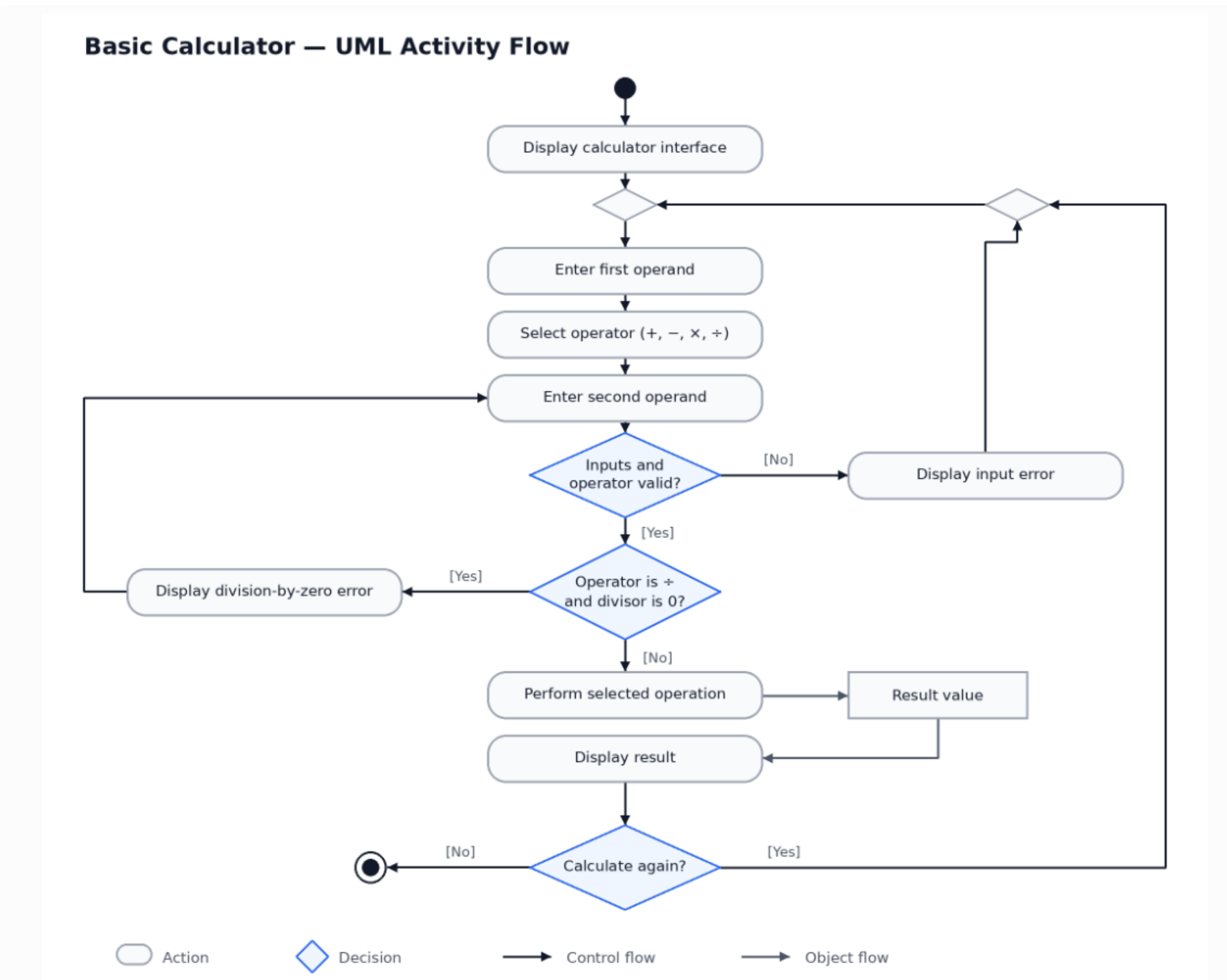




Decision vs. merge and **fork vs. join** each use the *same shape* — the difference is just how many arrows go in vs. out.

Actions should normally be named using a **verb–object** format:

- Display Calculator Interface
- Enter First Operand
- Select Operator
- Validate Inputs
- Perform Selected Operation
- Display Result



Explanation of the calculator diagram (this is an example of how you will explain the diagram)

Initial node

The solid black circle indicates that the process begins when the calculator interface is opened.

An initial node does not represent an activity. It only identifies where control enters the activity.

Display calculator interface

This is the first system action. It prepares the interface through which the user enters the operands and selects an operator.

Enter the operands and operator

The user supplies three important values:

- First operand;
- Selected operator; and
- Second operand.

For example:

First operand: 12

Operator: ÷

Second operand: 4

These values constitute the input required by the calculation action.

Validate the input

The system determines whether:

- Both operands are numeric;
- Required fields are completed;
- The operator is supported; and
- The input follows the required format.

This produces a decision:

[valid] → Continue processing

[invalid] → Display input error

If an input is invalid, the system displays an error and returns the user to data entry.

Check for division by zero

Even when the input is numeric, an additional rule applies to division:

operator = division AND second operand = 0

The decision has two guarded outcomes:

- [Yes] → Display Division-by-Zero Error
- [No] → Perform Selected Operation

This decision should appear after general input validation because checking a mathematical rule is meaningful only after the values have been confirmed as valid.

Perform selected operation

The system performs one of the following:

$$\text{Result} = \begin{cases} A + B & \text{if operator is addition} \\ A - B & \text{if operator is subtraction} \\ A \times B & \text{if operator is multiplication} \\ A \div B & \text{if operator is division} \end{cases}$$

This may be represented as one high-level action, **Perform Selected Operation**, when the diagram describes the general calculator process.

For a more detailed algorithm, it can be expanded using a decision node with four outgoing paths:

[operator = +] → Add Operands

[operator = −] → Subtract Operands

[operator = ×] → Multiply Operands

[operator = ÷] → Divide Operands

These alternative paths should subsequently be recombined through a merge node before displaying the result.

Result Value

The calculation produces a Result Value or output depending on the inputs entered.

Repeat decision

After displaying the result, the system asks whether the user wants another calculation.

- [Yes] returns to operand entry.
- [No] proceeds to the activity final node.

The loop must have an explicit exit condition that once the result is satisfied then process stops. This should not be interpreted that process continues indefinitely.

Activity final node

The bull's-eye symbol terminates the complete calculator activity. It is reached when the user chooses not to perform another calculation.